

# Linear Programming in 2D

- 1 Half Plane Representation of Polygons
  - numpy, scipy, and matplotlib
- 2 Linear Programming
  - geometric problem formulation
  - problem specification
- 3 Incremental 2D Linear Programming
  - adding constraints one by one
  - algorithm for 2D bounded LP

MCS 481 Lecture 11  
Computational Geometry  
Jan Verschelde, 18 September 2026

# Linear Programming in 2D

## 1 Half Plane Representation of Polygons

- numpy, scipy, and matplotlib

## 2 Linear Programming

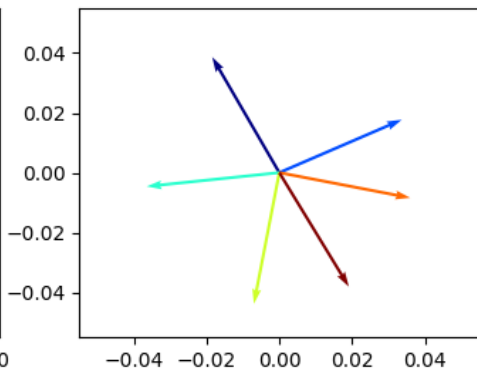
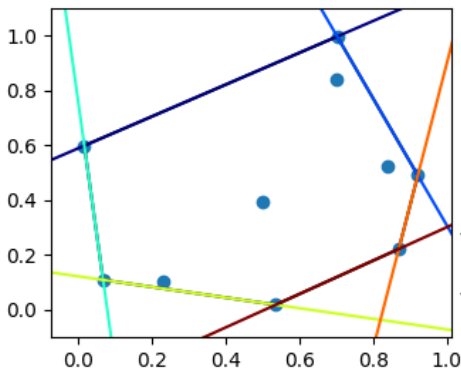
- geometric problem formulation
- problem specification

## 3 Incremental 2D Linear Programming

- adding constraints one by one
- algorithm for 2D bounded LP

# Half Plane Representation of Polygons

with numpy, scipy, and matplotlib



- At the left are all input points and the lines through all edges.
- At the right are the outward pointing normals to the edges.

# Linear Programming in 2D

## 1 Half Plane Representation of Polygons

- numpy, scipy, and matplotlib

## 2 Linear Programming

- geometric problem formulation
- problem specification

## 3 Incremental 2D Linear Programming

- adding constraints one by one
- algorithm for 2D bounded LP

# linear programming in $d$ variables

In standard form, a linear programming problem is given by

- 1 an objective function in  $d$  variables, and
- 2 a finite number  $n$  of constraints.

Maximize  
subject to

$$c_1x_1 + c_2x_2 + \dots + c_dx_d$$

$$a_{1,1}x_1 + a_{1,2}x_2 + \dots + a_{1,d}x_d \leq b_1$$

$$a_{2,1}x_1 + a_{2,2}x_2 + \dots + a_{2,d}x_d \leq b_2$$

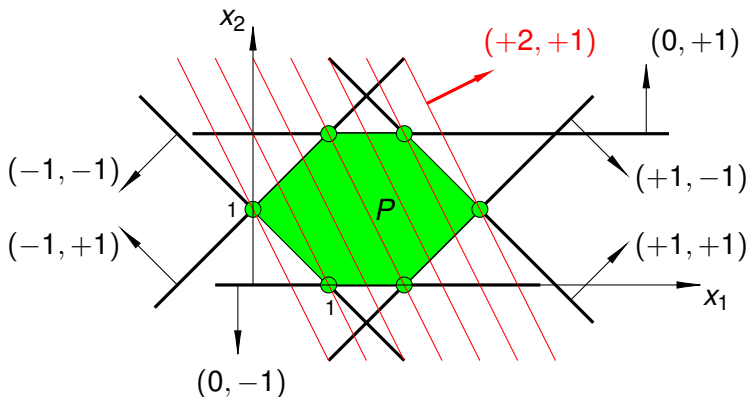
$$\vdots$$

$$a_{n,1}x_1 + a_{n,2}x_2 + \dots + a_{n,d}x_d \leq b_n$$

If we want only one solution to a problem, then writing the problem as a linear programming problem can get that one solution faster.

# geometric interpretation

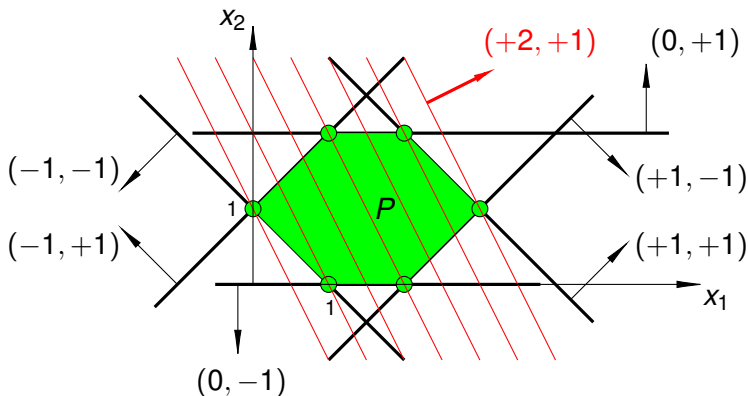
Maximize  $2x_1 + x_2$  subject to  $-x_2 \leq 0$ ,  $x_2 \leq 2$ ,  
 $-x_1 - x_2 \leq -1$ ,  $-x_1 + x_2 \leq 1$ ,  $x_1 - x_2 \leq 2$ ,  $x_1 + x_2 \leq 4$ .



Points on the same red line have the same value of  $2x_1 + x_2$ .

# geometric interpretation of half planes

Maximize  $2x_1 + x_2$  subject to  $-x_2 \leq 0$ ,  $x_2 \leq 2$ ,  
 $-x_1 - x_2 \leq -1$ ,  $-x_1 + x_2 \leq 1$ ,  $x_1 - x_2 \leq 2$ ,  $x_1 + x_2 \leq 4$ .



For all half planes  $a_1x_1 + a_2x_2 \leq b$ , with  $b = \max_{(x_1, x_2) \in P} a_1x_1 + a_2x_2$ .

# Linear Programming in 2D

## 1 Half Plane Representation of Polygons

- numpy, scipy, and matplotlib

## 2 Linear Programming

- geometric problem formulation
- **problem specification**

## 3 Incremental 2D Linear Programming

- adding constraints one by one
- algorithm for 2D bounded LP

# problem specification

On input are

- 1 the coefficients  $(c_1, c_2, \dots, c_d)$  of the objective function, and
- 2 a tuple  $H$  of  $n$  half planes  $H = (h_1, h_2, \dots, h_n)$ , given by
  - ▶ outward pointing normal vectors  $(a_{i,1}, a_{i,2}, \dots, a_{i,d})$ , and
  - ▶ corresponding bounds  $b_i$ , for  $i = 1, 2, \dots, n$ .

Let  $P = \bigcap_{i=1}^n h_i$ , the polyhedron defined by all half planes.

The output is the point  $(x_1, x_2, \dots, x_n) \in P$  at which the objective function  $c_1 x_1 + c_2 x_2 + \dots + c_d x_d$  is maximized.

Questions:

- 1 Does this problem always have a solution?
- 2 Is the solution unique?

# four cases

Answering the questions

- 1 Does this problem always have a solution?
- 2 Is the solution unique?

leads to the following case analysis:

- 1 The solution is *unique*.
- 2 The problem is *infeasible* if  $P = \emptyset$ .
- 3 The problem is *unbounded* if the optimal value is  $\infty$ .
- 4 The solution is *not* unique if the coefficient vector of the objective is parallel to one of the outward pointing normal vectors of  $P$ .

**Exercise 1:** If the coefficient vector of the objective is parallel to one of the outer normals of  $P$ , then the solution is not unique. Does the opposite direction hold as well? Can you prove the if and only if? Otherwise make a drawing why such a proof is not possible.

## making problems bounded with unique solution

To make the problem bounded, we can impose restrictions as follows.

Let  $M$  be some large enough number.

For a given objective  $(c_1, c_2, \dots, c_d)$ , add the constraints:

$$m_i = \begin{cases} x_i \leq M, & \text{if } c_i > 0, \\ -x_i \leq M, & \text{if } c_i \leq 0, \end{cases}$$

for  $i = 1, 2, \dots, d$ .

Then  $m_1 \cap m_2 \cap \dots \cap m_d$  is an orthogonal wedge.

In case the solution is not unique,  
then we choose the lexicographically smallest solution as *the* solution.

Any feasible program has thus a unique bounded solution,  
which is a vertex of  $P$ , called *the optimal vertex*.

# Linear Programming in 2D

## 1 Half Plane Representation of Polygons

- numpy, scipy, and matplotlib

## 2 Linear Programming

- geometric problem formulation
- problem specification

## 3 Incremental 2D Linear Programming

- adding constraints one by one
- algorithm for 2D bounded LP

# adding constraints one by one

Let  $H = (h_1, h_2, \dots, h_n)$  be the tuple of half planes, with the added constraints  $m_1$  and  $m_2$ .

Consider  $P_i$  as the region defined by

- ❶  $m_1$  and  $m_2$ , the added constraints, and
- ❷ the first  $i$  half planes of the tuple  $H$ .

Formally, for  $i = 0, 1, \dots, n$ :

$$P_i = m_1 \cap m_2 \cap h_1 \cap h_2 \cap \dots \cap h_i.$$

The regions are getting smaller as we add more constraints:

$$P_0 \supseteq P_1 \supseteq P_2 \supseteq \dots \supseteq P_n = P.$$

# updating the optimal vertex

For  $i = 0, 1, \dots, n$ : the region  $P_i = \bigcap_{j=1}^i h_j \cap m_1 \cap m_2$ ,

has the optimal vertex  $v_i$  with coordinates  $(x_{i,1}, x_{i,2})$ :

$$c_1 x_{i,1} + c_2 x_{i,2} = \max_{(x_1, x_2) \in P_i} c_1 x_1 + c_2 x_2.$$

## Lemma (update the optimal vertex)

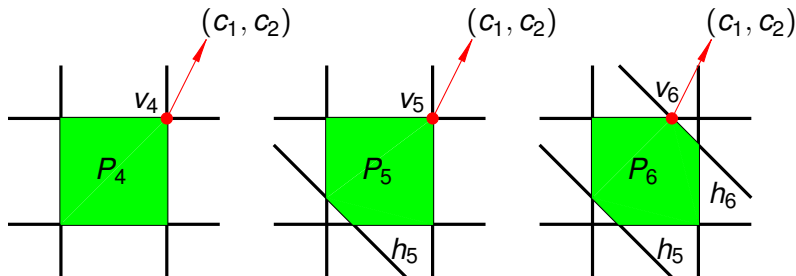
Consider adding  $h_i$  to  $P_{i-1}$  with current optimal vertex  $v_{i-1}$ .

- 1 If  $v_{i-1} \in h_i$ , then  $v_i = v_{i-1}$ .
- 2 If  $v_{i-1} \notin h_i$ , then either  $P_i = \emptyset$  or  $v_i \in \ell$ , the line bounding  $h_i$ .

## two cases illustrated

We start with four half planes and the optimal vertex  $v_4$ .

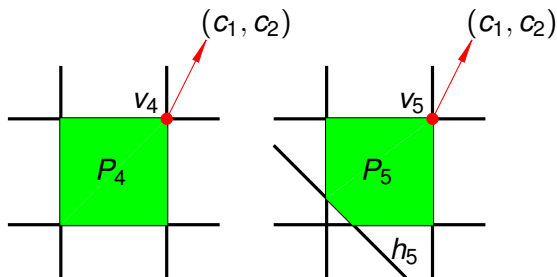
- 1 Adding  $h_5$  does not change the optimal vertex,  $v_5 = v_4$ .
- 2 Adding  $h_6$  changes the optimal vertex,  $v_6 \neq v_5$ .



Observe that  $v_6$  lies on the line of the half plane  $h_6$ .

## proof of the first case

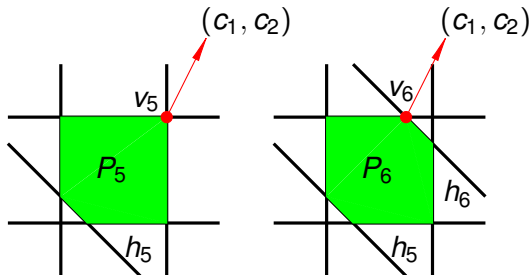
- 1 If  $v_{i-1} \in h_i$ , then  $v_i = v_{i-1}$ .



If  $v_{i-1} \in h_i$ , then  $v_{i-1}$  is the optimal vertex over  $P_i$ , as  $P_i \subseteq P_{i-1}$ , the maximum cannot increase over a smaller set, so  $v_i = v_{i-1}$ .

## proof of the second case

- ② If  $v_{i-1} \notin h_i$ , then either  $P_i = \emptyset$  or  $v_i \in \ell$ , the line bounding  $h_i$ .

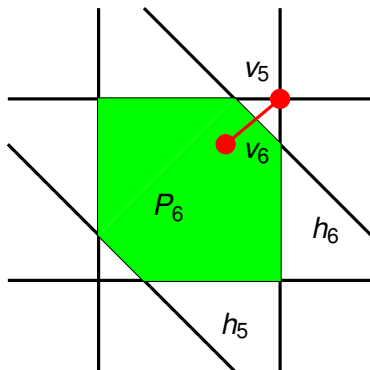


If  $P_i = \emptyset$ , then we are done. Assume  $P_i \neq \emptyset$ .

Assume  $v_6 \notin \ell$ , we will derive a contradiction.

# looking for a contradiction

Assume  $v_6 \notin \ell$ , we will derive a contradiction.  
Consider the line segment  $(v_{i-1}, v_i)$ .



$(v_{i-1}, v_i) \in P_{i-1}$ , as  $P_i \subseteq P_{i-1}$

As the objective is linear,  
moving from  $v_i$  to  $v_{i-1}$  increases  
the objective monotonically.

Consider  $q \in (v_{i-1}, v_i) \cap \ell$ .

$q$  exists because  $v_{i-1} \notin h_i$   
and  $v_i \in P_i$ .

As the objective is monotone  
increasing from  $v_i$  to  $v_{i-1}$ ,  
its value at  $q$  is higher than at  $v_i$ .

# Linear Programming in 2D

## 1 Half Plane Representation of Polygons

- numpy, scipy, and matplotlib

## 2 Linear Programming

- geometric problem formulation
- problem specification

## 3 Incremental 2D Linear Programming

- adding constraints one by one
- algorithm for 2D bounded LP

# linear programming in one dimension

If  $v_{i-1} \notin h_i$  and  $P_i \neq \emptyset$ , then  $v_i \in \ell$ , the line bounding  $h_i$ .

How to compute  $v_i$ ?

The line  $\ell$  has equation  $a_{i,1}x_1 + a_{i,2}x_2 = b_i$ ,  
we eliminate  $x_2$ :  $x_2 = (b_i - a_{i,1}x_1)/a_{i,2}$ .

This gives rise to a one dimensional problem:

Maximize  $c_1x_1 + c_2(b_i - a_{i,1}x_1)/a_{i,2}$   
subject to

$$a_{j,1}x_1 + a_{j,2}(b_i - a_{i,1}x_1)/a_{i,2} \leq b_j \\ j = 1, 2, \dots, i-1$$

Depending on the coefficient of  $x_1$ , every inequality gives either an upper or a lower bound, so we left with  $A \leq x_1 \leq B$ .

## Lemma (cost of one dimensional linear programming)

*Solving an LP problem in 1 variable with  $i$  inequalities takes  $O(i)$  time.*

# algorithm for 2D bounded LP

Algorithm 2DBOUNDEDLP( $H, c$ )

Input:  $H = (h_1, h_2, \dots, h_n)$ ,  $c = (c_1, c_2)$ .

Output:  $(x, y)$  optimal vertex or  $i$  if  $P_i = \emptyset$ .

- 1 set  $M$  constraints,  $v_0$  is corner of orthogonal wedge
- 2 for  $i$  from 1 to  $n$  do
- 3     if  $v_{i-1} \in h_i$  then
- 4          $v_i = v_{i-1}$
- 5     else if 1D LP is infeasible then
- 6         return  $i$
- 7     else
- 8          $v_i =$  solution of 1D LP
- 9 return  $v_n$

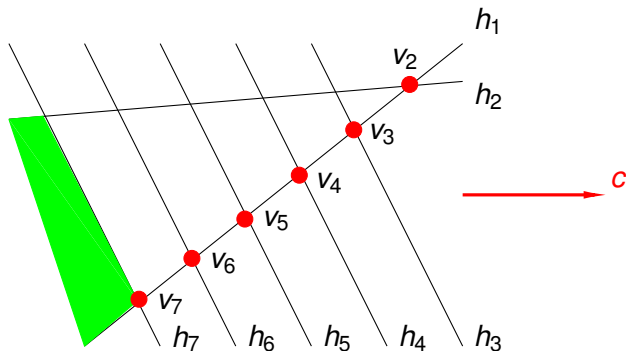
**Exercise 2:** Show the correctness of 2DBOUNDEDLP.

# cost of incremental linear programming

## Theorem (cost of incremental linear programming)

*Algorithm 2DBOUNDEDLP takes  $O(n^2)$  operations for  $n$  half planes.*

The proof uses  $\sum_{i=1}^n O(i)$  is  $O(n^2)$ .



## a computational exercise

**Exercise 3:** Use numpy, scipy, and matplotlib to define a linear programming problem.

- 1 Compute the convex hull of 10 random points in the plane.  
This convex hull will be the feasible region.
- 2 Use the equations of the lines through the edges to formulate the constraints of the linear programming problem.
- 3 Generate a random objective function.  
Order the vertex points along their value of the objective.
- 4 Make a plot similar to slide 7.

# summary and exercises

We covered section 4.3 in the textbook.

Consider the following activities, listed below.

- 1 Write your solutions to exercises 1, 2, and 3.
- 2 Consider the exercises 10,11,12 in the textbook.